

PACKT WORKSHOP · PART 1

Skills & Specs Lab

Building reliable AI agents for network operations:
from raw tools, to skills, to specs.

	TOOLS	raw capability
	SKILLS	scope + structure
	SPECS	the contract

William Collins



About William



- **Tech Evangelist, Podcaster, Community Builder**
- 20 years in networking and automation
- LinkedIn Learning Instructor
- Host of The Cloud Gambit Podcast (Packet Pushers)

Let's connect





What we'll cover

01

Why tools break down at scale

02

From tools to skills (Module 1)

03

Spec-driven development (Module 2)

04

Then: both labs, hands-on

All the theory first, then both labs, hands-on, back to back.

01

Why Tools Break Down at Scale

Tools expose capability. That is exactly the problem.

Three layers, one stack



Tools

Raw capability

"What can I do?"



Skills

Capability + scope + structure

"How do we do this job here?"



Specs

The contract for behavior

"What does correct look like, and how do we prove it?"

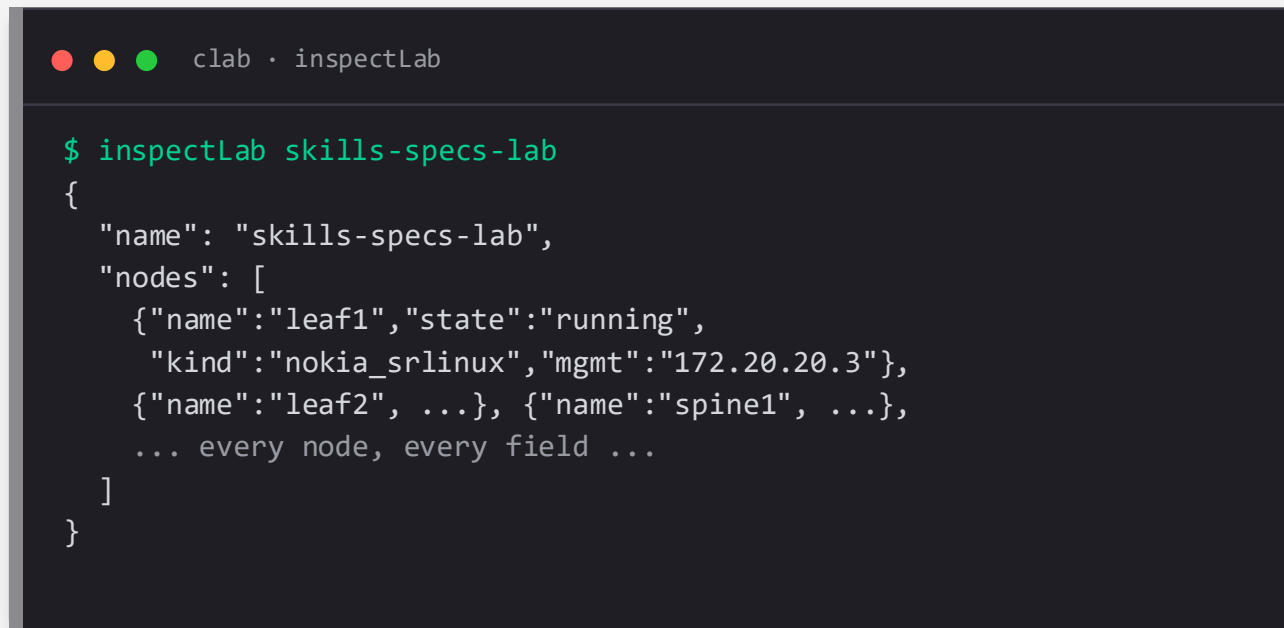
Today climbs this stack. Tools you have; skills and specs are the work.

A tool is exposed capability

A tool is a function the model can call.

Accurate. Unopinionated. Unscoped.

Example: the Containerlab MCP server exposes a read-only inspectLab tool. Ask it about the fabric and you get...

A terminal window with a dark background and light-colored text. The title bar shows three colored circles (red, yellow, green) followed by the text 'clab · inspectLab'. The terminal content shows a command prompt '\$' followed by the command 'inspectLab skills-specs-lab'. The output is a JSON object with a 'name' field and a 'nodes' array. The 'nodes' array contains three objects: 'leaf1' (state: 'running', kind: 'nokia_srlinux', mgmt: '172.20.20.3'), 'leaf2', and 'spine1'. The text '... every node, every field ...' is visible after the 'spine1' object, indicating a truncated output.

```
clab · inspectLab

$ inspectLab skills-specs-lab
{
  "name": "skills-specs-lab",
  "nodes": [
    {"name": "leaf1", "state": "running",
     "kind": "nokia_srlinux", "mgmt": "172.20.20.3"},
    {"name": "leaf2", ...}, {"name": "spine1", ...},
    ... every node, every field ...
  ]
}
```

Accurate. Also a firehose. It does not answer the question you asked.

...and where that breaks down



Firehose

Returns everything, answers nothing specific.



No scope

You interpret the raw output, every time.



No least privilege

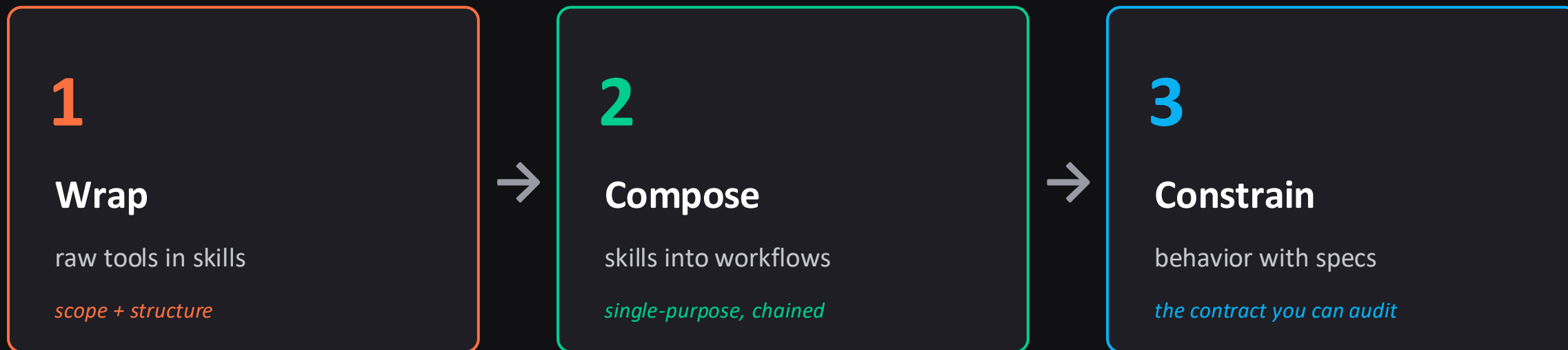
Every capability is in reach, including the dangerous ones.



Doesn't compose

Bolt three tools together and the glue is all yours.

The move: tools → skills → specs



We cover all the theory first, then build both labs back to back.

02

From Tools to Skills

Give capability scope, structure, and a defined answer.

A skill is a tool with a job

A tool's capability, wrapped in semantic structure, scope, and a defined output.



Semantic structure

Named fields and a verdict, not a blob.



Scope

One job. Clear in, clear out.



Defined output

The same shape on every call.

```
SKILL.md

---
name: network-state-query
description: focused,
  structured health for
  one fabric node
state: active
---

# Network state query
Ask the operational question
directly. Return a scoped,
labeled, structured result.
```

Same question, two experiences

"Is leaf1 healthy right now?"

RAW TOOL

```
inspectLab

$ inspectLab skills-specs-lab
{ "nodes": [
  { "name": "leaf1",
    "state": "running",
    "mgmt": "172.20.20.3"},
  ... every node ... ] }
→ container is up.
  interfaces? bgp? you decide.
```

SKILL

```
network-state-query leaf1

{
  "node": "leaf1",
  "interfaces": [{"ethernet-1/1": "up"}],
  "bgp": [{"peer": "10.1.1.1",
            "state": "established"}],
  "healthy": true,
  "summary": "leaf1 healthy: 4 up,
              all sessions established" }
```

What makes a skill worth keeping

1

One job per skill

Three jobs is three skills.

2

A defined output shape

Same structure every call; downstream can rely on it.

3

Named failure modes

Say what happens when a precondition fails.

4

Least privilege

Read-only is a scope decision, shown in config.

5

Graceful failure

Stop and say so. Never fake a success.

Skills are not free

“A one-off task that will not recur does not need a skill. Recognizing over-engineering early is part of the discipline.”

Build it

- Recurring job
- Defined output
- Reused by others

Don't

- One-off task
- Fuzzy scope
- Three jobs in a trench coat

Skills compose; tools don't



Two single-purpose skills, composed in one sentence, caught a dropped BGP session. No new code.

Skills that build skills



skills-creator

core skill

Scaffolds a new SKILL.md: trigger, scope, inputs / outputs, failure modes. The quality bar, enforced.



contributing

post-workshop

Walks the fork-and-PR flow, including least-privilege PAT setup. Your path after today.

Choosing what loads into your agent is itself the lesson.

03

Spec-Driven Development

From vibes to specs: behavior you can reason about.



**Agents that work in a demo fail in production,
because the expected behavior was never written
down.**

the whole reason for Module 2

A spec is the contract

Intent

The outcome, not the procedure.

Inputs

Exact, named values.

Constraints

Musts, and the must-nots.

Expected output

A shape, not a vibe.

Success criteria

One way to run it, one way to read it.

Failure modes

What 'not done' looks like. Never fake success.

Empty sections are where drift lives.

From a Slack message to a contract

service-delivery · request

Hey, we're standing up a small internal service on leaf1.

Put 10.50.0.1/32 on a new loopback and make sure it's reachable from the other leaf.

Let me know when it's done.



No routing stated

A loopback with an address is not automatically reachable.

"Reachable" from where?

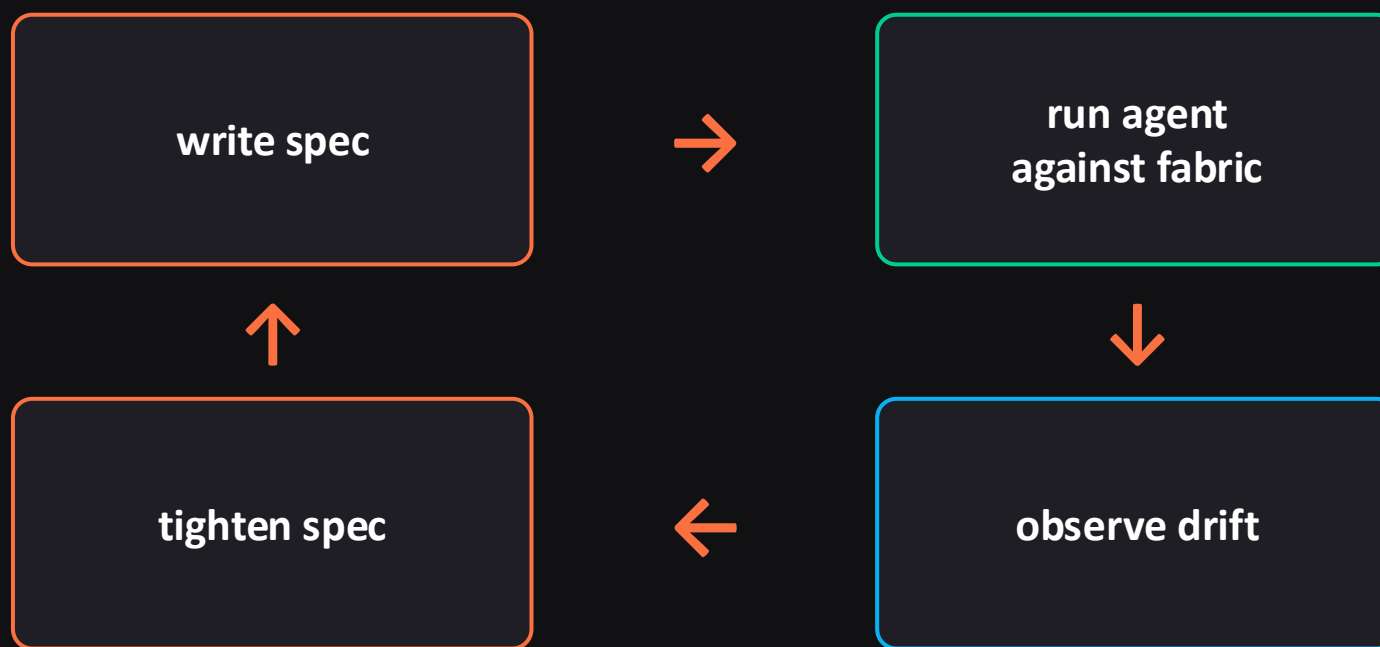
The real goal, with no way to confirm it named.

"Done" left to the agent

It will decide for itself what done means.

The spec's job is to close the gaps the request leaves open.

The loop



You change the contract, not the agent, its tools, or its prompt.

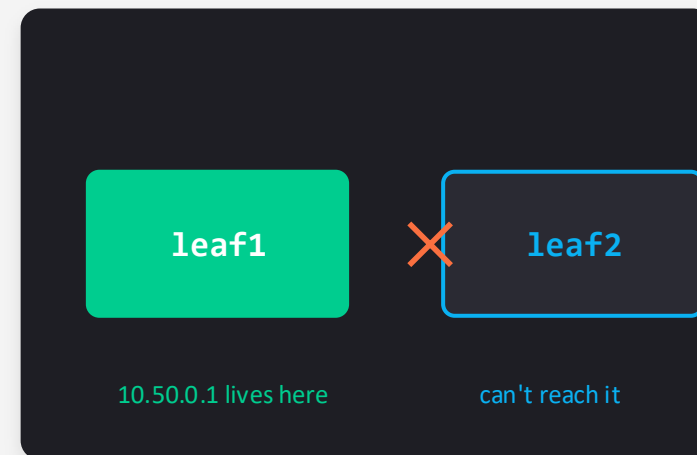
The drift is a trap, not luck

Gap 1: never advertised

The spec never requires lo0.0 in network-instance default, so the prefix is never advertised into the fabric.

Gap 2: vague success check

"Confirm it works" lets the agent pick the check that passes: a local ping on leaf1.



Agent reports success. From leaf2, the address is unreachable.

Determinism over cleverness: the trap reproduces every run.

Two additions remove the drift

SPEC v1 · drifts

spec-v1.md

Success criteria:

Confirm the new address works.

Constraints:

Don't disturb existing BGP.

→ advertisement optional

SPEC v2 · holds

spec-v2.md

Constraints:

lo0.0 MUST be in
network-instance default.

Success criteria:

ping from leaf2 returns
replies. Local ping is not
acceptable proof.

Same agent. Same tools. Same prompt. **Only the spec changed.**

Built the way we tell you to build agents

This workshop was specced before it was built. A constitution is the contract every build session worked against.

- The student never loses their lab
- Least privilege shown in config, not prose
- Determinism over cleverness
- Skills and specs are durable artifacts

spec/constitution.md

```
# Constitution:  
# Skills & Specs Lab
```

```
Article I  
  The student never loses  
  their lab.
```

```
Article IV  
  Determinism over  
  cleverness.
```



What to take home



Wrap tools in skills

Scope + structure + defined output.



Compose, don't bolt

Single-purpose skills into workflows.



Least privilege in config

The tools: filter is the lesson, not the prose.



Specs are the contract

Tighten behavior without touching the agent.

04

Now We Build

Theory's done. Both labs, hands-on, back to back.

NOW: HANDS-ON

Lab 1: Tools to Skills

1a

Raw tool vs skill, side by side

1b

Build device-state-query + change-validation

1c

Chain them against the live fabric

```
bash

$ gridctl link claudes-code
$ open lab-01/README.md

# confirm your client sees
# network-state-query
```

Broken at any point? ./scripts/reset.sh returns known-good in ~90s.

NOW: HANDS-ON

Lab 2: Spec-Driven Development

1 Read the request; run spec v1

2 Watch it drift, verify from leaf2

3 Tighten to v2, rerun, drift gone

bash · the real check

```
# run FROM leaf2:  
$ docker exec \  
  clab-...-leaf2 sr_cli \  
  "ping ... 10.50.0.1 -c 2"
```

Nobody tightens until everyone has seen the drift.



Where you stand at the midpoint

- ✓ Running Nokia SR Linux fabric
- ✓ gridctl stack up + client linked
- ✓ Two working skills in your registry
- ✓ One iterated, drift-free spec

```
checkpoint

$ ./scripts/checkpoint.sh

✓ fabric healthy
✓ skills present
✓ spec applied
midpoint: GREEN
```

Everyone crosses the midpoint green, then John takes it into agentic loops, OpenClaw, and NetClaw.

Thank you

RESOURCES

- › github.com/wcollins/skills-and-specs-lab
- › containerlab.dev
- › github.com/gridctl/gridctl
- › agentskills.io
- › github.com/github/spec-kit

Next: Part 2, with John Capobianco on agents in the real world.



William Collins

Packet Pushers – The Cloud Gambit Host